

# Linear Velocity Calibration

---

## Linear Velocity Calibration

1. Course Content
2. Preparations
  - 2.1 Content Description
  - 2.2 Start the Agent
3. Run the Example
  - 3.1 Start the Program
  - 3.2 Start Calibration
  - 3.3 Write Calibration Parameters to the Chassis
4. Code Explanation
  - 4.1 View Node Relationship Graph
  - 4.2 Source Code Analysis

## 1. Course Content

---

1. Learn how to calibrate the robot's linear velocity. After running the program, click "start" in the visualization interface, and the robot chassis will begin to move forward, stopping when the error is less than the tolerance value.

### [!IMPORTANT]

- The vehicle has been calibrated before leaving the factory. Under normal circumstances, recalibration is not necessary. If there is a significant error in the actual vehicle due to special circumstances, refer to this tutorial to calibrate the linear velocity.

## 2. Preparations

---

### 2.1 Content Description

This lesson uses the Jetson Orin NX as an example. For Raspberry Pi boards, you need to open a terminal on the host machine, enter the command to get into the Docker container, and then input the commands mentioned in this lesson in the terminal within the Docker container. A tutorial on how to enter the Docker container from the host can be found in this product's guide [00.Configuration and Operation Guide]-[3.Enter the Docker (For RPI5)]. For ORIN and RDK boards, you can directly open a terminal and enter the commands mentioned in this lesson.

### 2.2 Start the Agent

**Note: To test all examples, you must start the Docker agent first. If it is already running, you do not need to start it again.**

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:

```
Terminal
d: 0x000(1)
[6222.246556] info      | ProxyClient.cpp | create_datareader | datarea
der created | client_key: 0x77BF8684, datareader_id: 0x002(6), subscriber_id
: 0x002(4)
[6222.252366] info      | ProxyClient.cpp | create_topic      | topic c
reated      | client_key: 0x77BF8684, topic_id: 0x008(2), participant_id: 0x
000(1)
[6222.255974] info      | ProxyClient.cpp | create_subscriber | subscri
ber created | client_key: 0x77BF8684, subscriber_id: 0x003(4), participant_i
d: 0x000(1)
[6222.261071] info      | ProxyClient.cpp | create_datareader | datarea
der created | client_key: 0x77BF8684, datareader_id: 0x003(6), subscriber_id
: 0x003(4)
[6222.265578] info      | ProxyClient.cpp | create_topic      | topic c
reated      | client_key: 0x77BF8684, topic_id: 0x009(2), participant_id: 0x
000(1)
[6222.268942] info      | ProxyClient.cpp | create_subscriber | subscri
ber created | client_key: 0x77BF8684, subscriber_id: 0x004(4), participant_i
d: 0x000(1)
[6222.275467] info      | ProxyClient.cpp | create_datareader | datarea
der created | client_key: 0x77BF8684, datareader_id: 0x004(6), subscriber_id
: 0x004(4)
^C[ros2run]: Interrupt
```

## 3. Run the Example

Note:

- For the **Raspberry Pi** series, you need to enter the Docker container first.

### 3.1 Start the Program

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

Run the linear velocity calibration node:

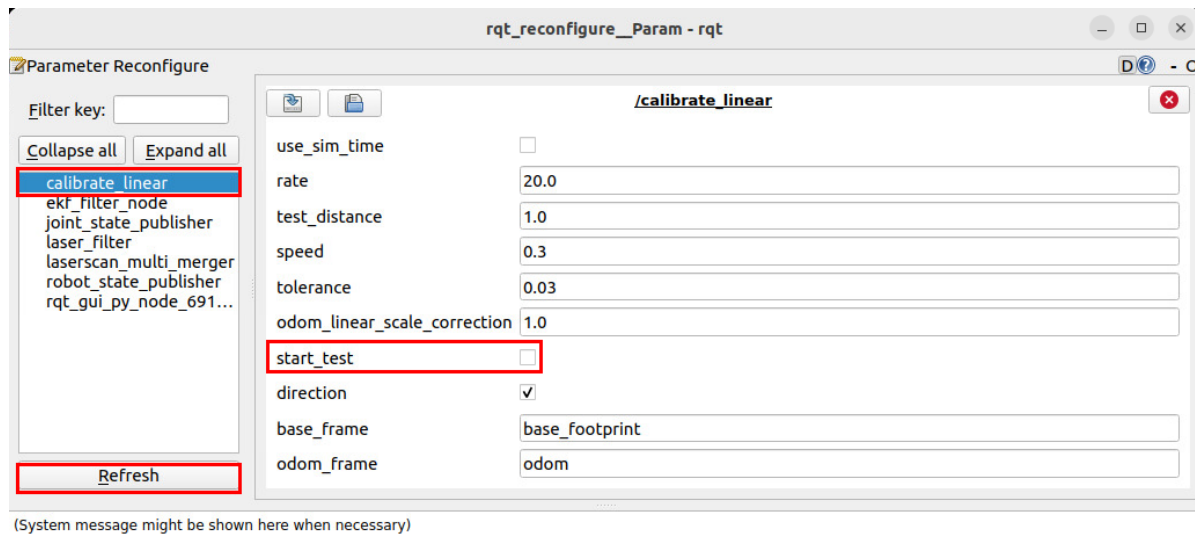
```
ros2 launch m3_bringup calibration.launch.py
```

```
jetson@yahboom: ~
jetson@yahboom: ~ - 160x21
jetson@yahboom:~$ ros2 launch m3_bringup calibration.launch.py
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2025-12-05-14-16-39-855114-yahboom-688385
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [launch.user]: =====Robot model ROSMASTER-M3 start=====
[INFO] [launch.user]: LASER start sucefully!,Start by independent node
[INFO] [launch.user]: =====Robot Base Start=====
[INFO] [joint_state_publisher-1]: process started with pid [688441]
[INFO] [robot_state_publisher-2]: process started with pid [688443]
[INFO] [lasers_merger-3]: process started with pid [688445]
[INFO] [scan_to_scan_filter_chain-4]: process started with pid [688447]
[lasers_merger-3] [INFO] [1764915400.445438227] [laserscan_multi_merger]: Waiting for topics ...
[scan_to_scan_filter_chain-4] [WARN] [1764915400.453193222] [rcl_logging_rosout]: Publisher already registered for provided node name. If this is due to multipl
e nodes with the same name then all logs for that logger name will go out over the existing publisher. As soon as any node with that name is destructed it will
unregister the publisher, preventing any further logs for that name from being published on the rosout topic.
[robot_state_publisher-2] [INFO] [1764915400.453508271] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-2] [INFO] [1764915400.453687412] [robot_state_publisher]: got segment base_link
[robot_state_publisher-2] [INFO] [1764915400.453712020] [robot_state_publisher]: got segment camera_link
[robot_state_publisher-2] [INFO] [1764915400.453724821] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-2] [INFO] [1764915400.453734581] [robot_state_publisher]: got segment laser23_frame
[robot_state_publisher-2] [INFO] [1764915400.453744085] [robot_state_publisher]: got segment lwheel1 Link
```

- Open the dynamic parameter panel, run in the terminal:

```
ros2 run rqt_reconfigure rqt_reconfigure
```

Click the **calibrate\_linear** node in the node options on the left:



#### [!WARNING]

- When you first open it, the above node may not be present. Click **Refresh** to see all the nodes. The displayed **calibrate\_linear** node is the one for calibrating linear velocity.

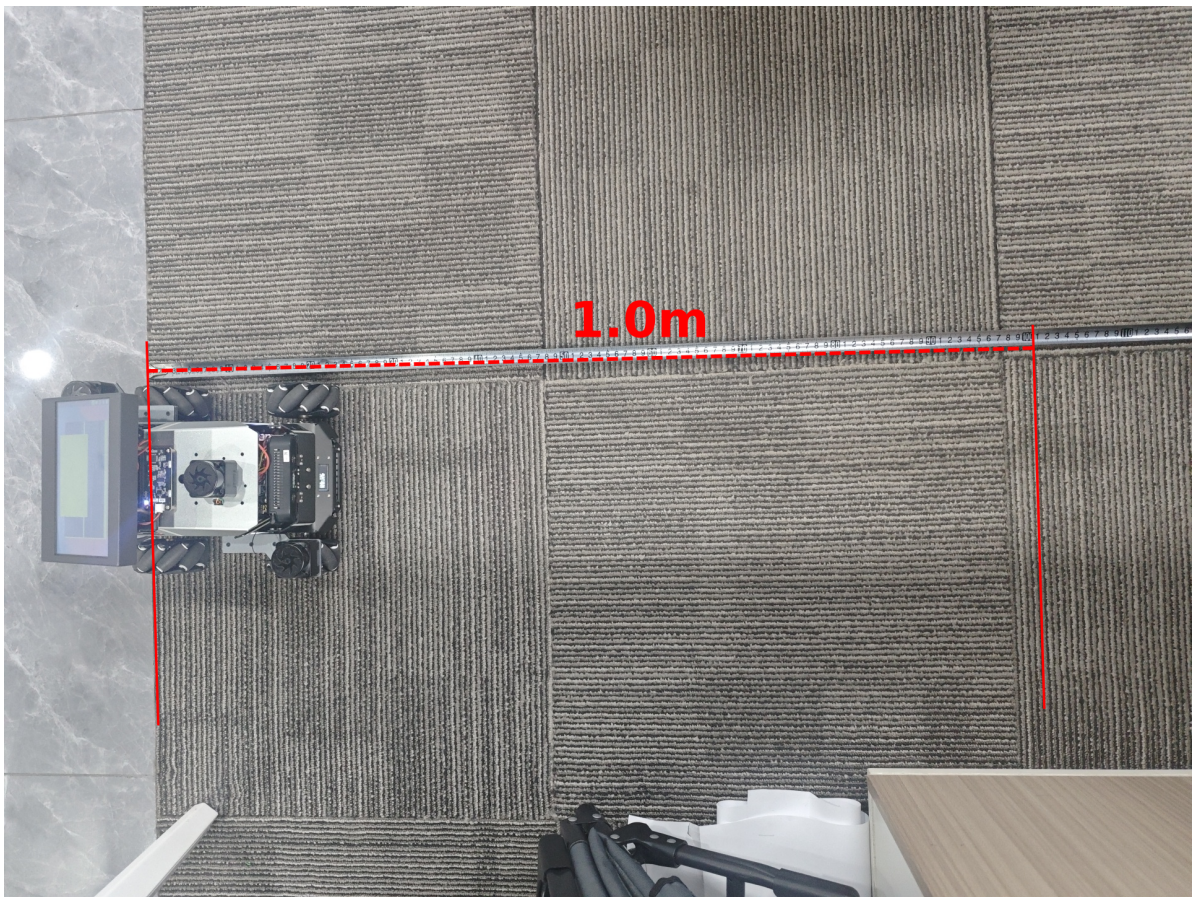
RQT interface parameter description:

- **test\_distance**: The distance for the calibration test. Here, it's set to move forward 1 meter.
- **speed**: The linear velocity.
- **tolerance**: The allowable error tolerance.
- **odom\_linear\_scale\_correction**: The linear velocity scale factor. If the test results are not ideal, modify this value.
- **start\_test**: The test switch.
- **direction**: Can be ignored. This value is for Mecanum wheel cars and can be modified to calibrate the linear velocity for side-to-side movement.
- **base\_frame**: The name of the base coordinate frame.
- **odom\_frame**: The name of the odometry coordinate frame.

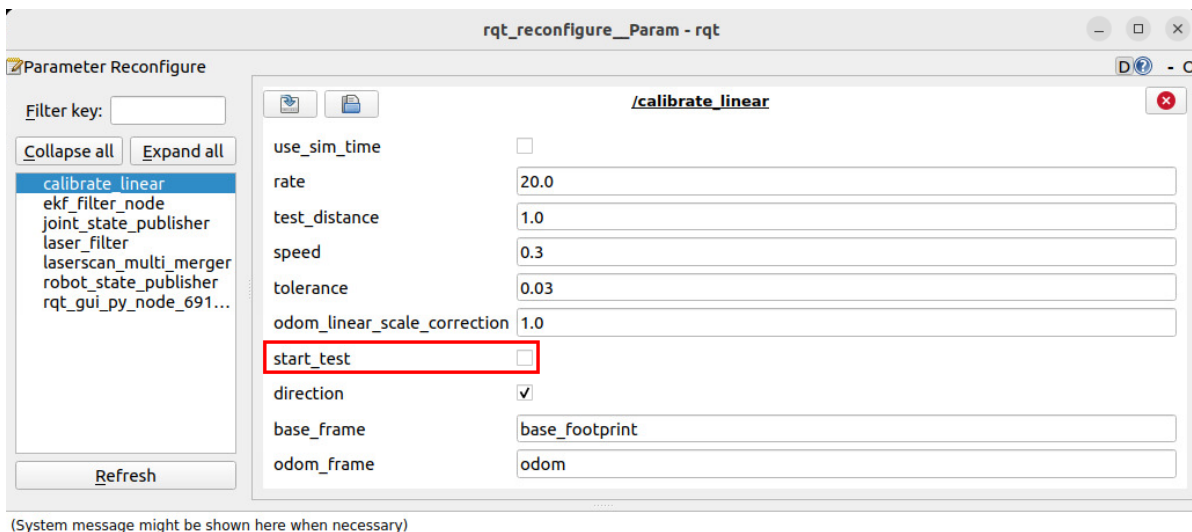
## 3.2 Start Calibration

1. Choose an object of **known length** on the ground as a reference (e.g., a measuring tape, floor tiles). Change **test\_distance** to the actual test distance. Here, we use a measuring tape for measurement, and you can use either the front or the back of the car as the measurement point.





2. Check the **start\_test** box to begin the calibration test.



3. After clicking **start\_test**, the calibration begins. The car will listen to the TF transform between **base\_footprint** and **odom**, calculate the theoretical distance traveled, and when the error is less than **tolerance**, it will publish a stop command and the terminal will print **done**.

- If the actual distance the car travels is less than 1m, increase the **odom\_linear\_scale\_correction** parameter appropriately. After modifying it, click on a blank area, then uncheck and recheck **start\_test** to reset it.



- Then, click `start_test` again to calibrate. The process is the same for modifying other parameters; you need to click a blank area to write the modified parameter. Record the final calibrated **odom\_linear\_scale\_correction** parameter.
4. Repeat the above steps until you find a suitable **odom\_linear\_scale\_correction** parameter.

```

jetson@yahboom: ~
jetson@yahboom: ~ 160x21
[robot_state_publisher-2] [INFO] [1764915400.453772310] [robot_state_publisher]: got segment rwheel2_Link
[joint_state_publisher-1] [INFO] [1764915401.055495159] [joint_state_publisher]: Waiting for robot_description to be published on the ro
[scan_to_scan_filter_chain-4] [WARN] [1764915401.446608358] [laser_filter]: diagnostic_updater: No HW_ID was set. This is probably a bug
devices that do not have a HW_ID, set this value to 'none'. This warning only occurs once all diagnostics are OK. It is okay to wait unt
fore calling setHardwareID.
[lasers_merger-3] [INFO] [1764915401.447731045] [laserscan_multi_merger]: Subscribing to topics 2
[INFO] [launch.user]: =====Merge_Odom Localization Start=====

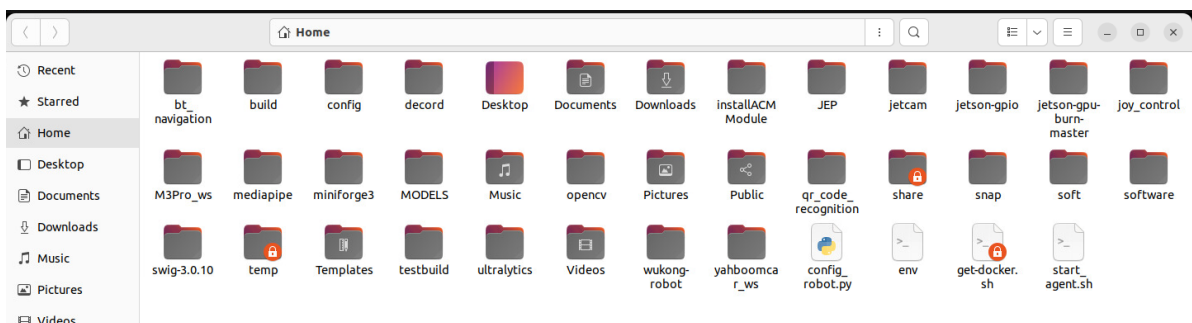
[INFO] [ekf_node-5]: process started with pid [688576]
[INFO] [calibrate_linear-6]: process started with pid [688701]
[calibrate_linear-6] [INFO] [1764916520.234076894] [calibrate_linear]: distance: 0.03602202378497022,error: -0.9639779762150298
[calibrate_linear-6] [INFO] [1764916520.668757716] [calibrate_linear]: distance: 0.18329221266398862,error: -0.8167077873360113
[calibrate_linear-6] [INFO] [1764916521.170085028] [calibrate_linear]: distance: 0.32560594350519867,error: -0.6743940564948013

[calibrate_linear-6] [INFO] [1764916521.669665445] [calibrate_linear]: distance: 0.4875625621670932,error: -0.5124374378329069
[calibrate_linear-6] [INFO] [1764916522.170039516] [calibrate_linear]: distance: 0.6296410133815937,error: -0.37035898661840627
[calibrate_linear-6] [INFO] [1764916522.668813927] [calibrate_linear]: distance: 0.7877198023968806,error: -0.21228019760311945
[calibrate_linear-6] [INFO] [1764916523.160723860] [calibrate_linear]: distance: 0.9433540380256303,error: -0.056645161074271786
[calibrate_linear-6] [INFO] [1764916523.270089919] [calibrate_linear]: done
[calibrate_linear-6] [INFO] [1764916523.271096347] [calibrate_linear]: distance: 0.9747343573755523,error: -0.02526564262444775

```

### 3.3 Write Calibration Parameters to the Chassis

- To write parameters to the chassis, you must first disconnect the chassis agent by pressing **Ctrl+C** or closing the chassis connection agent terminal.
- Open the **config\_robot.py** file in the home directory on the vehicle.



- Uncomment line 551, fill in the previous calibration coefficient in the parentheses of **robot.set\_ros\_scale\_line(xx)**, and then click **save**.

```

config_robot.py
538 car_type = robot.read_car_type()
539 if car_type is not None:
540     print("car_type:", car_type)
541     if int(car_type) == robot.CAR_TYPE:
542         break
543     time.sleep(.5)
544
545 # robot.set_ros_domain_id(30)
546 # robot.set_ros_namespace("")
547 # robot.set_motor_pid_parm(0.0, 0.06, 0.5)
548 # robot.set_imu_yaw_pid_parm(0.0, 0, 0.3)
549
550 robot.set_ros_scale_line(0.89)
551 #robot.set_ros_scale_angluar(1.0)
552
553

```

- Open a terminal on the vehicle and enter the command:

```
python3 config_robot.py
```

```
jetson@yahboom: ~  
jetson@yahboom:~$ python3 config_robot.py  
Read device: Linux  
Waiting to read the car type  
car_type: 7  
version: 1.0.6  
domain_id: 17  
ros_namespace:  
ros_scale_line: 0.890  
ros_scale_angluar: 1.000  
motor pid parm: 0.800, 0.060, 0.500  
imu pid parm: 0.600, 0.000, 0.300  
arm_mid: 2000,1949,1981,1984,1486,3100  
jetson@yahboom:~$
```

Wait for the parameters to be written. The `ros_scale_line:0.890` in the terminal's printed information is the written parameter. The chassis linear velocity calibration is complete.

## 4. Code Explanation

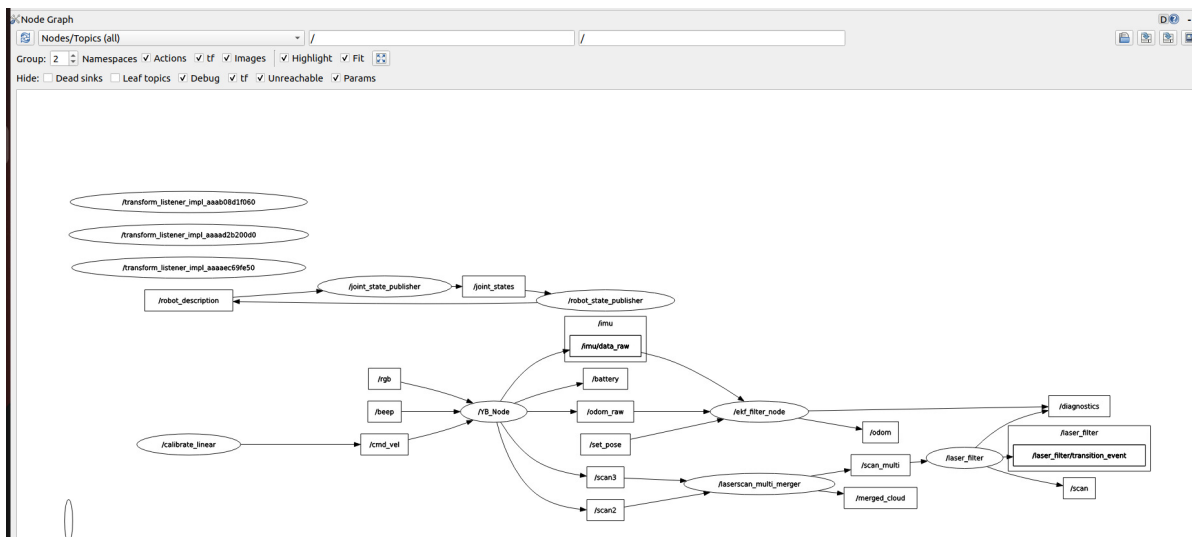
Source code path:

```
/home/jetson/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/calibration/
calibrate_linear.py
```

## 4.1 View Node Relationship Graph

Open a terminal and enter the command:

```
ros2 run rqt_graph rqt_graph
```



In the node relationship diagram above:

- The **/ekf\_filter\_node** subscribes to the chassis's raw odometry **/odom\_raw** and IMU data **/imu/data\_raw**, and after data fusion, it publishes the **/odom** topic.
- The **calibrate\_linear** node listens to the TF transform from **odom** to **base\_footprint** and publishes the **/cmd\_vel** topic to control the robot chassis movement.

## 4.2 Source Code Analysis

Listening to the TF coordinate transformation is implemented in the `get_position` method of the `CalibrateLinear` class:

```
def get_position(self):
    try:
        now = rclpy.time.Time()
        transform = self.tf_buffer.lookup_transform(
            self.base_frame,
            self.odom_frame,
            now,
            timeout=rclpy.duration.Duration(seconds=1.0))
        return transform

    except (LookupException, ConnectivityException, ExtrapolationException):
        self.get_logger().info('transform not ready')
        raise
```

Judging the robot chassis's displacement and controlling its movement is implemented in the `on_timer` method (timer callback function) of the `CalibrateLinear` class:

```
def on_timer(self):
    move_cmd = Twist()
    #self.get_param()
    self.start_test =
self.get_parameter('start_test').get_parameter_value().bool_value
    self.odom_linear_scale_correction =
self.get_parameter('odom_linear_scale_correction').get_parameter_value().double_value
    self.direction =
self.get_parameter('direction').get_parameter_value().bool_value
    self.test_distance =
self.get_parameter('test_distance').get_parameter_value().double_value
    self.tolerance =
self.get_parameter('tolerance').get_parameter_value().double_value
    self.speed = self.get_parameter('speed').get_parameter_value().double_value
    if self.start_test:
        '''trans = self.tf_buffer.lookup_transform(
            self.odom_frame,
            self.base_frame,
            now,
            )'''

        self.position.x = self.get_position().transform.translation.x
        self.position.y = self.get_position().transform.translation.y

        self.get_logger().info(f"self.position.x: {self.position.x}")
        self.get_logger().info(f"self.position.y: {self.position.y}")

        distance = sqrt(pow((self.position.x - self.x_start), 2) +
                        pow((self.position.y - self.y_start), 2))
        distance *= self.odom_linear_scale_correction
        # print("distance: ",distance)
        self.get_logger().info(f"distance: {distance}")
```



```

error = distance - self.test_distance
# print("error: ",error)
self.get_logger().info(f"error: {error}")
#start = time()
if abs(error) < self.tolerance:
    self.start_test =
rcipy.parameter.Parameter('start_test',rcipy.Parameter.Type.BOOL,False)
    all_new_parameters = [self.start_test]
    self.set_parameters(all_new_parameters)
    self.get_logger().info("done")
else:
    if self.direction:
        print("x")
        move_cmd.linear.x = copysign(self.speed, -1 * error)
    else:
        move_cmd.linear.y = copysign(self.speed, -1 * error)
        print("y")
    self.cmd_vel.publish(move_cmd)
#end = time()
else:
    self.x_start = self.get_position().transform.translation.x
    self.y_start = self.get_position().transform.translation.y
    self.get_logger().info(f"self.x_start: {self.x_start}")
    self.get_logger().info(f"self.y_start: {self.y_start}")

    self.cmd_vel.publish(Twist())

```